

Understanding Unix

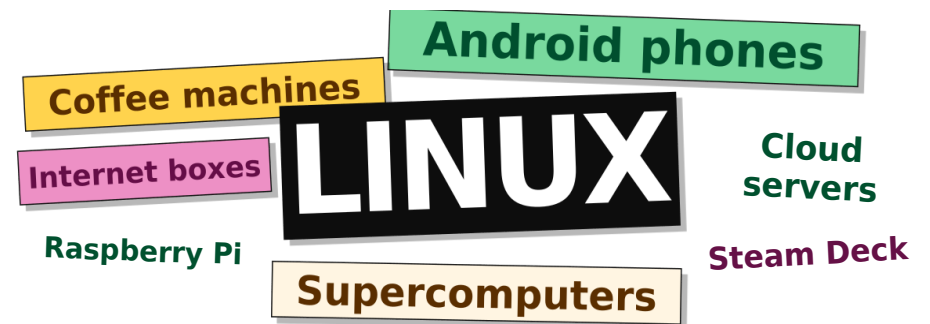
NUnix · L3 MIAGE · CM1 · 90 minutes

ISTIC · University of Rennes 2026–2027



1/50

Linux is closer than you think



You can use it all day without opening a terminal.

2/50

Yes, even the coffee machine



Android phones

The **Linux kernel** manages processes, memory and hardware.

Google · platform architecture



Freebox Server Pop

A home internet box. Free publishes its **Linux source code**.

Free · firmware 4.9.10



MARO Model One

A coffee machine with **embedded Linux**, remote support and updates.

Toradex · May 2026

Linux can run behind a touchscreen or a web interface.

3/50

Unix and Linux usage



Mobile web use

69.17%

of observed mobile page views come from **Android**, built on Linux.

Statcounter · Sept 2026

Websites

92.1%

use the **Unix family**, including Linux, among sites whose OS W3Techs identifies.

W3Techs · 1 Oct 2026

Supercomputers

500/500

systems in the **June 2026 TOP500 list** use the Linux family.

TOP500 · June 2026

4/50

Course structure



Three lectures of 90 minutes.

Lecture	Topics	Application
CM1 · Unix	Files, paths and permissions	Find files and check access
CM2 · Commands	Processes, search, pipes and redirections	Count errors in a log
CM3 · Bash	Variables, conditions, loops and functions	Write scripts with arguments and loops

The exercises practise commands first, then scripts.

Working with files in Unix



- 1. Navigate directories and understand file paths.
- 2. Read files and quote names containing spaces.
- 3. Check file and directory permissions.

Why use a script?



Today: find the main error in **one log**.
Next week: check **200 logs every morning**, including when you are away.

A saved script lets the team:

- ▶ Apply the same rule to every input.
- ▶ Review and change the procedure as text.
- ▶ Run it remotely or through a scheduler.
- ▶ Check output and failure status.

7/50

Which tool fits the task?



Task	A useful starting point
Explore sales figures, filter and make a chart	Excel or another graphical analysis tool
Inspect a server’s logs, processes and permissions	Remote shell and system commands
Repeat a file-processing procedure every morning	A tested script and a scheduler
Query related business tables	SQL and a database
Explain the result to the team	A spreadsheet, dashboard or report

Excel also offers Power Query and Office Scripts for repeatable work.

8/50

1. Linux and the shell



The operating system, the terminal and the command interpreter

What the operating system manages



Resource	Operating system responsibilities
Processes	Start, schedule and stop programs
Memory	Allocate memory and isolate processes
Files	Store data and control access
Devices	Provide interfaces to hardware

Programs request services through **system calls**.

Linux



- ▶ **1991:** Linus Torvalds announces his kernel project.
- ▶ Linux is a **Unix-like kernel**, distributed as free and open source software.
- ▶ A distribution combines the kernel with tools, libraries and applications.

Our working environment: a Linux system with the **Bash shell** and command-line tools.

11/50

Unix, GNU and Linux



Name	Meaning
Unix	The operating-system family that inspired Linux
GNU	The project behind Bash and many command-line tools
Linux	The kernel: manages processes, memory and hardware

GNU = GNU's Not Unix.

The name contains itself: a recursive acronym.

12/50

Terminal, shell, command



Name	Role
Terminal emulator	Displays text and receives keyboard input
Shell	Interprets command lines and runs commands
Command	Performs an operation, such as listing files

A **script** saves a sequence of shell commands in a file.

A command and its arguments



```
ls -l /home/alice
```

Part	Role
ls	Command name
-l	Option: request the long listing format
/home/alice	Argument: directory to list

Arguments provide information to a command when it starts.

Getting help



```
man ls
info coreutils
apropos directory
help cd
```

- ▶ **Tab** completes a name or displays possible completions.
- ▶ **Up** recalls a previous command. `history` lists history.
- ▶ `clear` clears the display. In `man`, **q** quits.

15/50

One directory tree



- ▶ The filesystem has a common root: **/**.
- ▶ Other filesystems appear at **mount points** in that tree.
- ▶ Each account has a **home directory** and belongs to groups.
- ▶ Permissions control access to files and directories.

Example home directory: `/home/alice`
The university may use a different path.

16/50

Where Linux stores files



Need	Typical location
Alice's personal files	/home/alice
System configuration	/etc
Installed add-on applications	/opt
Changing application data	/var/lib
Log files	/var/log

Configuration: /etc · Logs: /var/log

17/50

Users, administrators and software



Directory	Typical role
/home, /root	User homes; the root account's home
/usr	Most distribution-provided tools and libraries
/usr/local	Software installed locally by the administrator
/opt	Add-on software packages

/ is the filesystem root.
root is also the administrator account's name.

18/50

Temporary files and system information



Directory	Purpose / lifetime
/var	Changing data, often persistent
/tmp	Temporary files; cleanup policy varies
/run	Runtime state, cleared at boot
/dev	Device interfaces
/proc, /sys	Views of processes, kernel and devices

A path can expose live system information.
It need not refer to ordinary data stored on disk.

19/50

Several filesystems, one tree



```
/
├── home/      user files
├── mnt/       a temporary mount point
└── media/     removable media mount points
```

Mounting makes another filesystem visible at a directory.
The device node might be /dev/sdb1; its files appear at the mount point.

20/50

The current directory



Notation	Meaning
.	Current working directory
..	Parent directory
/	Root directory
~	Your home directory, expanded by the shell
\$HOME	Variable containing your home path
\$PWD	Variable containing the current directory path

Absolute and relative paths



Current directory: /home/alice

Path	Interpretation
/home/alice/notes.txt	Absolute: start at /
notes.txt	Relative: start in the current directory
./notes.txt	The same relative path
../bob/notes.txt	Go to the parent, then into bob

A path tells us **where to look**. Permissions determine whether access is allowed.

2. Files and directories



Navigate, inspect and change the directory tree

23/50

Moving around



```
pwd  
cd /home/alice  
cd ..  
cd
```

- ▶ pwd prints the current working directory.
- ▶ cd path changes the current directory.
- ▶ cd or cd ~ returns to your home.

24/50

Listing directory entries



```
ls
ls -l
ls -a
ls -la /home/alice
```

- ▶ -l adds details, including ownership and permissions.
- ▶ -a includes names beginning with ., such as .bashrc.
- ▶ ls directory lists that directory's contents.

25/50

Creating and removing directories



```
mkdir practice
rmdir practice
```

rmdir removes an **empty** directory.

rm -r practice also removes the directory's contents.
Shell deletion does **not** send files to a recycle bin.

26/50

Creating, copying and moving files



```
touch notes.txt
cp notes.txt notes-copy.txt
mv notes-copy.txt archive.txt
rm archive.txt
```

touch creates a missing file, or updates timestamps of an existing file.

27/50

Reading text files



Command	Result
cat notes.txt	Print the whole file
less notes.txt	Browse interactively, q to quit
more notes.txt	Browse a page at a time
head -n 5 notes.txt	Print the first five lines
tail -n 5 notes.txt	Print the last five lines
diff old.txt new.txt	Show differences between two files

28/50

Why is less called less?



Mark Nudelman was investigating errors in large logs.
He wanted to read the lines **before** an error.

His version of more could not move backward.
The new pager's name joked about a **backward more**.

```
less app.log
```

29/50

Concatenating files



```
cat first.txt second.txt  
cat
```

- ▶ With filenames, cat prints their contents **in argument order**.
- ▶ Without filenames, cat reads **standard input**.
- ▶ At an empty terminal input line, **Ctrl+D** indicates end of input.

30/50

Filename patterns



Pattern	Matches
?	Exactly one character
*	Zero or more characters
[abc]	One character from the set

```
ls /etc/rc*
ls /etc/rc????
ls [abc]oo.c
```

The **shell** expands these patterns before running `ls`.

Quoting special characters



Form	Effect
*	Treat the next * literally
'*.txt'	Preserve every character inside the quotes
"\$HOME"	Expand the variable, keep the result as one word

```
echo *.txt
echo '*.txt'
cat "meeting notes.txt"
```

Quotes help preserve **one argument** even when its value contains spaces.

What does the command receive?



In a directory containing only a.txt and meeting notes.txt:

```
printf '<%s>\n' *.txt  
printf '<%s>\n' '*.txt'
```

```
<a.txt>  
<meeting notes.txt>  
<*.txt>
```

33/50

Checkpoint: files and paths



1. What does `cd ..` do?
 2. Which `ls` option includes hidden entries?
 3. What does `cat a.txt b.txt` print first?
 4. Does `echo '*.txt'` expand the pattern?
1. Move to the parent directory.
 2. `-a`
 3. The contents of `a.txt`.
 4. No. Single quotes preserve the literal `*`.

34/50

3. Who may access this file?



Identify the account, inspect the path, explain the permission check

35/50

Users, groups and ownership



```
whoami  
id  
ls -ld .  
ls -l
```

- ▶ `id` shows your user identity and groups.
- ▶ `ls -l` shows the file's owner, group and permission bits.
- ▶ `ls -ld` inspects a directory itself.

36/50

Permission classes



```
-rwxr-xr--
u  g  o
```

Class	Applies to
u	The file's owner
g	Members of the file's group
o	Other users
a	All three classes when using chmod

What r, w and x allow



Bit	Regular file	Directory
r	Read contents	List entry names
w	Modify contents	Add or remove entries*
x	Execute	Search / traverse

*Changing directory entries also requires search permission. Other restrictions, such as the sticky bit, may apply.

Changing permission bits



```
chmod u+x script.sh
chmod go-w notes.txt
chmod u=rw,g=r,o= notes.txt
```

- ▶ + adds the selected permission bits.
- ▶ - removes them.
- ▶ = sets the selected class to the listed permissions.

39/50

Numeric permissions



Each class adds its bit values: **r = 4, w = 2, x = 1.**

Owner	Group	Others
rwX = 7	rw- = 6	r-- = 4

```
chmod 764 notes.txt
```

The three digits describe **owner, group, others**, in that order.

40/50

Checkpoint: permissions



- | | |
|--|----------------------------------|
| 1. Decode 640 for a regular file. | 1. Owner: rw- |
| 2. Add execute permission for the owner of <code>run.sh</code> . | Group: r-- |
| 3. Which directory bit allows traversal? | Others: --- |
| | 2. <code>chmod u+x run.sh</code> |
| | 3. <code>x</code> |

41/50

"I can see the name, but cannot read it"



```
# Illustrative server metadata
drwxr----- alice students shared/
-rw-r----- alice students shared/notes.txt
```

Bob belongs to students. He can list the names, but cannot open `notes.txt`. Why?

The directory's group bits lack **x**: traversal.
If team access is intended, the owner can add `g+x`.

42/50

CM1 summary: files and permissions



1. Locate configuration, software, data and logs.
2. List and read files; quote names containing spaces.
3. Check owner, group and other permissions.

43/50

Optional reference



Examples to revisit during TD or personal practice

44/50

Operating system, shell and GUI



Layer	What it gives us
Operating system	Processes, storage and access control
Graphical interface	Windows, menus and direct visual interaction
Shell	Commands we can combine and save in scripts
Application	A task-specific tool, such as Excel or a database

Your laptop can run Windows while the company’s application runs on Linux. You need to work with both.

Summarise a log before opening it in Excel



Quantity	Meaning
2,000,000 log lines	Example input size
1,048,576 rows	The maximum in one Excel worksheet
One row per error code	The summary the team wants to discuss

Process the raw log where it lives, save the counts, then use Excel to analyse or present the summary.

Microsoft: worksheet limits. Power Query and the Data Model offer other ways to handle larger datasets.

Path components and word counts



```
dirname /home/alice/notes.txt
basename /home/alice/notes.txt
wc -w notes.txt
```

- ▶ `dirname` prints `/home/alice`.
- ▶ `basename` prints `notes.txt`.
- ▶ `wc -w` counts **words**. `wc -l` counts newline characters.

47/50

Hard links and symbolic links



Hard link

```
ln notes.txt other.txt
```

- ▶ Another name for the **same file**.
- ▶ Removing one name leaves the other usable.
- ▶ Same filesystem, normally no directory links.

Symbolic link

```
ln -s notes.txt shortcut
```

- ▶ Stores a **path** to a target.
- ▶ May cross filesystems or target a directory.
- ▶ Can become dangling if the target path disappears.

48/50

Builtins and external commands



Builtin

Implemented inside the shell.

```
cd ..
```

Can change the current shell's state.

External program

Loaded from an executable file.

```
ls -l
```

The shell searches directories in PATH.

49/50

Recognising a command



```
type cd
type ls
type -a echo
help cd
```

type reports how Bash resolves a name.
help documents its builtins.

50/50